PQC Migration Toolkit

User Handbook

A complete guide to understanding and using the Post-Quantum Cryptography Migration Toolkit

Written for students, developers and anyone curious about the future of encryption

No technical background required

Built with Python | kyber-py | dilithium-py | cryptography | matplotlib
100% free and open-source

Table of Contents

1. What is This Project?
2. Why Should You Care About Quantum Computers?
3. Understanding Encryption (Simple Explanation)
4. What Are Post-Quantum Cryptography Standards?
5. Project Structure - Where Everything Lives
6. How to Install and Run
7. Using the GUI Desktop App (Recommended)
8. Using the Command Line (main.py)
9. Module Guide: ML-KEM Key Exchange
10. Module Guide: ML-DSA Digital Signatures
11. Module Guide: Hybrid TLS Handshake
12. Module Guide: Certificate Migration
13. Module Guide: Performance Benchmarks
14. Module Guide: Hybrid Certificate Chain
15. Module Guide: Attack Demo (Harvest Now Decrypt Later)
16. Module Guide: Wire-Format Inspection
17. Module Guide: Migration Impact Calculator
18. Understanding the Results
19. Frequently Asked Questions
20. Troubleshooting Common Issues
21. Glossary of Terms
22. References and Further Reading

1. What is This Project?

The PQC Migration Toolkit is a hands-on educational software package that lets you explore the new encryption standards designed to protect against quantum computers. Think of it as a playground where you can experiment with the future of internet security.

The project contains six interactive modules that demonstrate different aspects of post-quantum cryptography (PQC). You can run them either through a Windows desktop application with buttons and tables, or through a command-line menu.

Everything is built using Python and runs entirely on your own computer. No internet connection, cloud accounts or API keys are needed. All the cryptographic operations happen locally.

2. Why Should You Care About Quantum Computers?

Most people do not realize this but the encryption that protects your bank transactions, WhatsApp messages, emails and passwords is based on math problems that quantum computers will be able to solve easily.

Here is a simple way to understand it. Current encryption is like a giant maze that would take a normal computer thousands of years to solve. A quantum computer is like a helicopter that can fly straight over the maze and reach the center in seconds.

The scary part is that hackers are already collecting encrypted data today. They are waiting for quantum computers to become powerful enough so they can decrypt everything later. This is called Harvest Now Decrypt Later. Your messages that you send today could be read years from now.

That is why the world needs to upgrade its encryption before quantum computers arrive. The new standards are called Post-Quantum Cryptography or PQC for short.

3. Understanding Encryption (Simple Explanation)

Imagine you want to send a secret message to your friend across a crowded room. Everyone can see the message so you put it inside a locked box. Only your friend has the key to open it.

That is exactly what encryption does. It locks your data so that only the person with the right key can read it.

There are three main concepts you need to understand:

Public Key

This is like a lock that anyone can use. You can make many copies and give them to everyone. Anyone can use this lock to lock a message for you.

Private Key

This is the unique key that opens the lock. You keep this secret. Only you have it. When someone sends you a locked message you use your private key to open it.

Digital Signature

This is like a wax seal on an envelope. It proves that the message really came from you and that nobody tampered with it on the way.

4. What Are Post-Quantum Cryptography Standards?

Post-Quantum Cryptography refers to new mathematical problems that are hard for both regular computers AND quantum computers to solve. These are the replacement for today's RSA and ECDH algorithms.

In August 2024 after an eight-year worldwide competition the National Institute of Standards and Technology (NIST) finalized three new standards:

FIPS 203 - ML-KEM (Key Encapsulation)

Also known as Kyber. This is a new way for two parties to exchange a secret key securely. It replaces RSA and ECDH key exchange. It comes in three security levels: ML-KEM-512 (128-bit), ML-KEM-768 (192-bit) and ML-KEM-1024 (256-bit).

FIPS 204 - ML-DSA (Digital Signatures)

Also known as Dilithium. This is a new way to sign and verify digital messages. It replaces RSA and ECDSA signatures. It comes in three levels: ML-DSA-44 (128-bit), ML-DSA-65 (192-bit) and ML-DSA-87 (256-bit).

FIPS 205 - SLH-DSA (Backup Signatures)

Also known as SPHINCS+. This is a backup standard in case the lattice-based algorithms (ML-KEM and ML-DSA) are broken in the future. It uses a completely different mathematical approach.

5. Project Structure - Where Everything Lives

All project files are located in the E:/pqc-migration-toolkit folder. Here is what each file does:

```
E:pqc-migration-toolkit/
  gui_app.py        The Windows desktop app (recommended)
  main.py           Command-line version with 9 modules
  app.py            Web dashboard version (requires Flask)
  build_exe.py      Build standalone .exe with PyInstaller
  requirements.txt  Package dependencies
  src/pqc_wrapper.py        Unified PQC API wrapper
  src/hybrid_cert.py        Real hybrid X.509 certs
  src/attack_demo.py        Harvest Now Decrypt Later demo
  src/wire_format.py        Hex dump with annotations
  src/impact_calc.py        Storage/bandwidth calculator
  src/kem_demo.py           Standalone ML-KEM demo
  src/sign_demo.py          Standalone ML-DSA demo
  src/hybrid_tls_demo.py    Standalone hybrid TLS demo
  src/cert_migration_demo.py  Certificate analysis
  src/benchmark.py          Performance benchmark suite
  tests/test_all.py   pytest test suite (18 tests)
  .github/workflows/ci.yml  GitHub Actions CI
  PQC_Handbook.pdf   This handbook you are reading
  PQC_Migration_Toolkit.pptx  Presentation slides
  PQC_Migration_Toolkit.pdf   Presentation in PDF format
  benchmarks/        Contains generated charts
```

6. How to Install and Run

Follow these steps to get the toolkit running on your computer.

Step 1 - Check Python

Open a Command Prompt or Terminal and type: python --version. You need Python version 3.9 or higher. If you do not have Python installed download it from python.org.

Step 2 - Install Dependencies

Type the following command and press Enter:

```
pip install kyber-py dilithium-py matplotlib cryptography
```

This will download and install all the required libraries. The process usually takes less than a minute.

Step 3 - Navigate to the Project Folder

```
cd E:pqc-migration-toolkit
```

Step 4 - Launch the App

For the desktop app (recommended):

```
python gui_app.py
```

For the command-line version:

```
python main.py
```

7. Using the GUI Desktop App (Recommended)

The GUI desktop app is the easiest way to use this toolkit. It looks and feels like a normal Windows program with buttons and tables.

When you run python gui_app.py a window will open with six tabs at the top:

How to use each module:

1. Click on a tab to select the module you want to explore.

2. Click the Run button that appears in that tab.

3. Wait a few seconds for the results to appear in tables or text boxes.

4. Read the output. Each module shows key sizes, timing and pass/fail status.

Tips:

- The app runs everything on your computer so it may take a few seconds for benchmarks.

- You can switch between tabs anytime. Each tab remembers its results.

- If the app freezes wait a moment. Heavy operations like key generation take a second.

- Close the window or press Ctrl+C in the terminal to exit.

8. Using the Command Line (main.py)

If you prefer a simple text interface you can use the command-line version instead.

Run the program:

```
python main.py
```

You will see a menu like this:

```
+-----------------------------------------------------+
|          PQC MIGRATION TOOLKIT - MAIN MENU          |
+-----------------------------------------------------+
|  1.  ML-KEM Key Encapsulation Demo   (FIPS 203)     |
|  2.  ML-DSA Digital Signature Demo   (FIPS 204)     |
|  3.  Hybrid TLS 1.3 Handshake Demo                  |
|  4.  Certificate Migration Analysis                  |
|  5.  Performance Benchmarks + Charts                |
|  6.  Hybrid Certificate Chain (RSA + ML-DSA)        |
|  7.  Attack Demo: Harvest Now Decrypt Later         |
|  8.  Wire-Format Inspection (hex dump)              |
|  9.  Migration Impact Calculator                    |
|  10. Run ALL modules sequentially                   |
|                                                     |
|  0.  Exit                                           |
+-----------------------------------------------------+
```

Simply type the number of the module you want to run and press Enter. Option 7 runs all six modules one after another which is useful for demonstrations.

9. Module Guide: ML-KEM Key Exchange

This module demonstrates how two parties can establish a shared secret key using quantum-safe cryptography. It runs all three security levels (ML-KEM-512, 768, 1024) and displays the key sizes.

What you will see:

1. Open the app and click the ML-KEM Key Exchange tab.
2. Click the Run button.
3. A table appears showing public key size, private key size, ciphertext size and PASS/FAIL status.
4. If all three rows show PASS then key exchange is working correctly.
5. Notice that higher security levels have larger keys.

10. Module Guide: ML-DSA Digital Signatures

This module generates quantum-safe digital signatures at three security levels. It signs a test message and verifies the signature to confirm authenticity.

What you will see:

1. Open the ML-DSA Signatures tab.
2. Click Run.
3. The table shows public key, private key and signature sizes for each level.
4. All three should show VALID status.
5. Compare the signature sizes. ML-DSA-87 signatures are about 4.6KB which is much larger than classical signatures.

11. Module Guide: Hybrid TLS Handshake

This module simulates how modern browsers (like Google Chrome) establish a secure connection using both classical and quantum-safe cryptography together. This is called the hybrid approach.

What you will see:

1. Open the Hybrid TLS tab.
2. Click Run.
3. Read through the step-by-step output in the text box.
4. Look for the MATCH status at the end.
5. The output explains why using both algorithms together is safer than using just one.

12. Module Guide: Certificate Migration

This module compares the sizes of classical certificates (RSA, ECDSA) with post-quantum key sizes. It helps you understand one of the biggest challenges of upgrading the internet.

What you will see:

1. Open the Certificate Migration tab.
2. Click Run.
3. The table shows RSA-2048, RSA-4096 and ECDSA P-256 certificate sizes in bytes.
4. Below that you see ML-DSA-65 and ML-DSA-87 key sizes.
5. Notice that PQC keys are 3 to 10 times larger than classical keys.

13. Module Guide: Performance Benchmarks

This module measures how fast each cryptographic operation runs on your computer. It runs each operation three times and shows the average time in milliseconds.

What you will see:

1. Open the Benchmarks tab.
2. Click Run.
3. Two tables appear. One for ML-KEM timing and one for ML-DSA timing.
4. KeyGen is the time to create a key pair.
5. Sign is the time to sign a message (usually the slowest operation).
6. For reference a human blink takes about 300 milliseconds.

14. Module Guide: Hybrid Certificate Chain

This module generates a three-level certificate chain Root CA to Intermediate to Leaf using REAL X.509 certificates with ML-DSA public keys embedded as custom extensions. Each certificate carries BOTH a classical RSA signature AND an ML-DSA signature.

What you will see:

1. Open the Hybrid Cert Chain tab.
2. Click Run.
3. Read the step-by-step output showing each certificate.
4. Each cert has an RSA signature AND an ML-DSA signature embedded.
5. Both signatures are verified independently showing VALID status.
6. This is the exact model enterprises will use for PQC migration.

15. Module Guide: Attack Demo (Harvest Now Decrypt Later)

This module demonstrates why PQC migration is urgent. It simulates the Harvest Now Decrypt Later attack: hackers record encrypted traffic today, wait for quantum computers, and decrypt it later.

What you will see:

1. Open the Attack Demo tab.
2. Click Run.
3. Alice sends a secret message encrypted with small RSA.
4. Eve records the ciphertext now (Harvest Now).
5. Later Eve factors the RSA modulus and decrypts (Decrypt Later).
6. The demo then shows that ML-KEM-768 prevents this attack entirely.
7. The message is recovered successfully showing the vulnerability.

16. Module Guide: Wire-Format Inspection

This module shows the actual bytes of PQC keys, ciphertexts, and signatures in a hex dump with annotations. You can see the internal structure of each cryptographic artifact.

What you will see:

1. Open the Wire Format tab.
2. Click Run.
3. A hex dump of ML-KEM-768 public key appears with byte offsets.
4. Below that is the ciphertext, ML-DSA public key, and signature.
5. Each section shows the internal structure (seed, polynomials, challenge, vector).
6. A size comparison table shows all PQC artifacts side by side.

17. Module Guide: Migration Impact Calculator

This module estimates the real-world cost of switching to post-quantum cryptography. You can see how much extra storage and bandwidth PQC migration will require.

What you will see:

1. Open the Impact Calc tab.
2. Click Run.
3. The calculator shows certificate sizes for classical, hybrid, and PQC-only.
4. Storage impact for all certificates is calculated.
5. Bandwidth impact per TLS handshake is shown.
6. Cumulative extra bandwidth over months and years is estimated.
7. The conclusion: the bandwidth cost is generally acceptable for modern networks.

18. Understanding the Results

Here is what the different statuses and numbers mean:

PASS / VALID / MATCH

The operation completed successfully. For KEM this means both parties derived the same secret key. For signatures this means the signature was verified correctly. For TLS this means the client and server computed the same session key.

FAIL / INVALID / MISMATCH

Something went wrong. This should never happen in normal operation. If you see this restart the module and try again.

Key Sizes (bytes)

One byte is roughly one character of text. An RSA-2048 public key is 256 bytes or about the length of a tweet. An ML-KEM-1024 public key is 1568 bytes or about half a page of text.

Timing (milliseconds)

One millisecond is one thousandth of a second. A blink takes about 300 milliseconds. Most PQC operations complete in 2 to 50 milliseconds which is faster than a blink. Signing is usually the slowest operation.

19. Frequently Asked Questions

Is this safe to use for real encryption?

No. The pure Python libraries used in this toolkit are for educational purposes only. They are not constant-time and may have side-channel vulnerabilities. For real security use liboqs (the C library) or OpenSSL 3.5.

Why are PQC keys so much larger than RSA keys?

PQC algorithms are based on different mathematics (lattice problems) that require more data to achieve the same security level. RSA-2048 has 256-byte keys while ML-KEM-1024 has 1568-byte keys. This is one of the main challenges of migration.

Can I use this on Mac or Linux?

Yes. The Python code is cross-platform. Run the same commands on any operating system. The GUI app uses tkinter which works on all platforms.

Do I need internet to run this?

No. Once you install the dependencies everything runs locally on your computer. No internet connection is required.

What if I see errors about missing modules?

Run the install command again: pip install kyber-py dilithium-py matplotlib cryptography. Make sure you have Python 3.9 or newer.

How long does each module take to run?

KEM and Signature modules take about 2-3 seconds. TLS and Certificate modules take about 5 seconds. Benchmarks take about 30 seconds because they run each operation multiple times.

20. Troubleshooting Common Issues

'pip' is not recognized as a command

Python is not installed or not added to PATH. Download and install Python from python.org. During installation check the box that says Add Python to PATH.

'No module named kyber-py' error

You forgot to install the dependencies. Run: pip install kyber-py dilithium-py matplotlib cryptography

GUI window opens but is blank

Wait a few seconds for the interface to load. If it stays blank try running python gui_app.py from a terminal to see any error messages.

The app freezes when I click Run

This is normal. Cryptographic operations take a moment to compute especially benchmarks. Wait 5-10 seconds for the results to appear. The app will unfreeze once the operation completes.

I see a DeprecationWarning in the terminal

These warnings are harmless and do not affect functionality. They come from the Python libraries and can be ignored.

Permission denied when installing packages

Try adding --user to the pip command: pip install --user kyber-py dilithium-py matplotlib cryptography

21. Glossary of Terms

Certificate

A digital document that proves the identity of a website or organization. Contains a public key and is signed by a trusted authority.

Ciphertext

The encrypted output. The locked box that contains the secret message.

Decapsulation

The process of unlocking a shared secret using a private key in a KEM scheme.

Digital Signature

A cryptographic proof that a message was created by a specific sender and has not been altered.

Encapsulation

The process of locking a shared secret using a public key in a KEM scheme.

FIPS

Federal Information Processing Standards. Official US government standards for cryptography.

KEM

Key Encapsulation Mechanism. A method for two parties to establish a shared secret key.

KeyGen

The process of generating a new cryptographic key pair (public and private).

Lattice

A mathematical structure used in PQC algorithms. Think of it as a grid of points in multi-dimensional space.

ML-KEM

Module-Lattice-Based Key Encapsulation Mechanism. The NIST standard for quantum-safe key exchange.

ML-DSA

Module-Lattice-Based Digital Signature Algorithm. The NIST standard for quantum-safe signatures.

NIST

National Institute of Standards and Technology. The US agency that standardized PQC algorithms.

PQC

Post-Quantum Cryptography. Cryptographic algorithms designed to resist quantum computer attacks.

Public Key

A key that can be shared openly. Used to encrypt messages or verify signatures.

Private Key

A secret key that must be kept confidential. Used to decrypt messages or create signatures.

Quantum Computer

A computer that uses quantum mechanics to solve certain problems exponentially faster than classical computers.

RSA

A classical encryption algorithm based on the difficulty of factoring large numbers. Vulnerable to quantum computers.

TLS

Transport Layer Security. The protocol that secures HTTPS websites and encrypted communications.

22. References and Further Reading

Beginner-Friendly Resources:

NIST PQC Website: nist.gov/pqc

Cloudflare PQC Learning Center: cloudflare.com/learning/ssl/post-quantum-cryptography

YouTube: Post-Quantum Cryptography in 5 Minutes

YouTube: Quantum Computers Explained by Kurzgesagt

Official Standards Documents:

NIST FIPS 203 ML-KEM Standard: csrc.nist.gov/pubs/fips/203/final

NIST FIPS 204 ML-DSA Standard: csrc.nist.gov/pubs/fips/204/final

NIST FIPS 205 SLH-DSA Standard: csrc.nist.gov/pubs/fips/205/final

Libraries Used in This Project:

kyber-py (pure Python ML-KEM): github.com/GiacomoPope/kyber-py

dilithium-py (pure Python ML-DSA): github.com/GiacomoPope/dilithium-py

Open Quantum Safe (liboqs C library): openquantumsafe.org

OpenSSL 3.5 (PQC support in production): openssl.org

Who Already Uses PQC:

- Google Chrome (2024+) - X25519MLKEM768 hybrid key exchange
- Signal Messenger (2023+) - PQXDH protocol with Kyber
- Apple iMessage (2024+) - Post-quantum encryption
- Amazon Web Services (2020+) - PQC in AWS KMS
- Cloudflare - PQC available to all customers

Thank you for using the PQC Migration Toolkit.

The future of encryption starts with understanding it today.